CodiGo
powered by Tecsup

Curso: BACKEND

Tema: Test Unitarios con Junit y Mockito

CODIGO powered by Tecsup

Al finalizar esta clase, el participante será capaz de:

- Comprender los conceptos de las Test Unitarios
- Comprender sobre Junit, Mockito.
- El estudiante será capaz de realizar Test Unitarios para cobertura los métodos implementados.

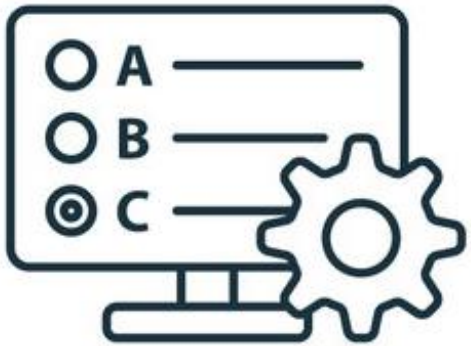
Temario:

1. Qué es un Test Unitario?
2. Tipos de Pruebas Unitarias
3. Que es JUnit?
4. Que es Mockito?
5. Que es JACOCO?
6. Implementando Junit & Mockito en Maven
7. Implementando Test Unitarios
8. Ejecución de Ejercicio
9. Practica en Clase
10. Concluciones

1. Qué es un Test Unitario?

Test Unitarios

Las pruebas unitarias consisten en aislar una parte del código y comprobar que funciona a la perfección. Son pequeños tests que validan el comportamiento de un objeto y la lógica.



TESTING

CodiGo
powered by Tecsup

Aislamiento de unidades de código

Integración continua

Retroalimentación temprana

Automatización

Documentación viva

Repetibilidad

2. Tipos de Pruebas Unitarias

Tipos de Pruebas Unitarias

- Pruebas de casos de éxito (happy path)
- Pruebas de casos de borde (boundary testing)
- Pruebas de casos de error (error testing)
- Pruebas de rendimiento (Jmeter)
- Pruebas de seguridad
- Pruebas de cobertura de código (code coverage)
- Pruebas de mockintegracións y stubs
- Pruebas de

JUnit

Es una de las herramientas más populares y ampliamente utilizadas para escribir y ejecutar pruebas unitarias en aplicaciones Java. Las pruebas unitarias son un enfoque de desarrollo de software que implica probar unidades individuales de código (como métodos o funciones) para asegurarse de que funcionen correctamente de manera aislada

Anotaciones: JUnit utiliza anotaciones de Java (como @Test, @Before, @After, @BeforeClass, @AfterClass, etc.)

Assertions (Afirmaciones): JUnit proporciona una variedad de métodos de aserción (por ejemplo, **assertEquals, assertTrue, assertFalse**, etc.)

Mockito

biblioteca de Java que se utiliza para simplificar y automatizar la creación de objetos simulados o "mocks" en el contexto de pruebas unitarias. Estos objetos simulados se utilizan para aislar la unidad de código bajo prueba de las dependencias externas, como clases, interfaces o servicios, de modo que puedas centrarte en probar la unidad de código de manera aislada y controlada

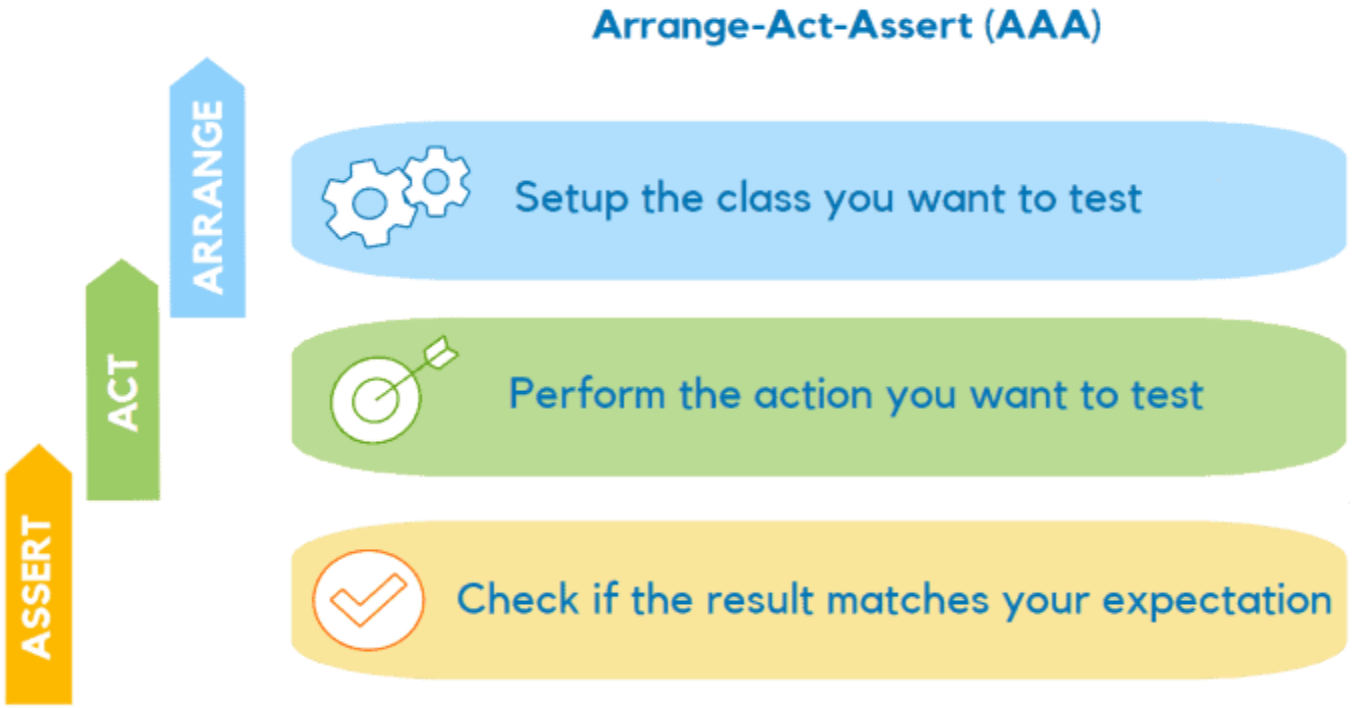
JACOCO (Java Code Coverage)

es una herramienta de código abierto utilizada para medir la cobertura de código en aplicaciones Java. La cobertura de código es una métrica que indica qué porcentaje del código fuente de un programa ha sido ejecutado por un conjunto de pruebas. Es una métrica importante en el proceso de pruebas unitarias y pruebas de integración, ya que ayuda a determinar cuán exhaustivas son las pruebas y cuántas partes del código no se han ejecutado.

JACOCO
Java Code Coverage

CODIGO powered by Tecsup

PATRON AAA



Implementando Junit & Mockito en Maven

Para poder usar Junit & Mockito debemos añadir las dependencias en el POM

```
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.7.0</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.7.0</version>
    <scope>test</scope>
</dependency>
```

Implementando Jacoco

Para poder usar Jacoco debemos añadir el plugin y ejecución del reporte.

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.7</version>
    <executions>
        <execution>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>prepare-package</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
    </executions>
```

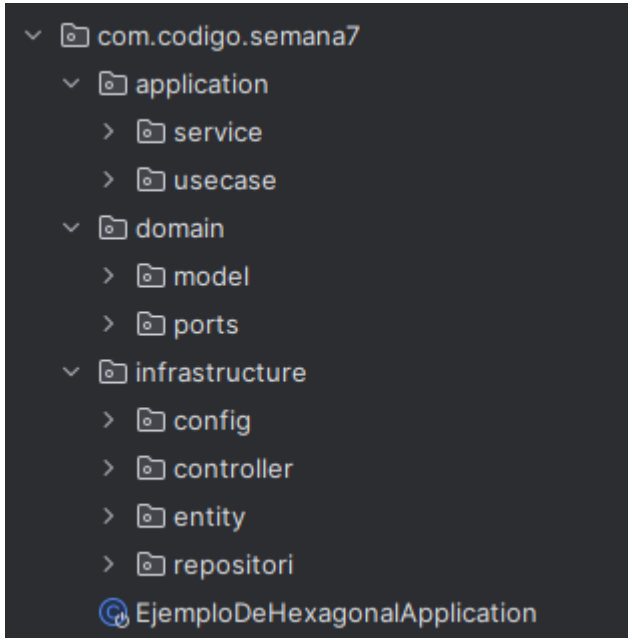
Implementando Exclusiones Jacoco

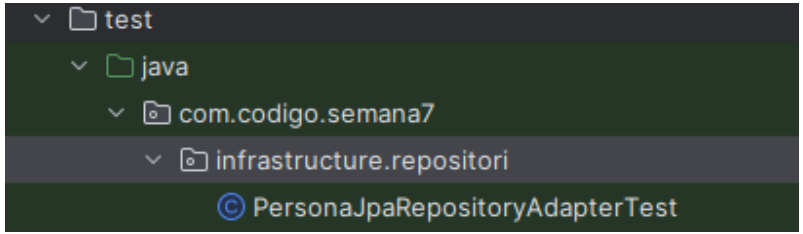
Para poder excluir alguna clase o paquete, debemos realizar la configuración de lo que se desea excluir.

```
<configuration>
    <excludes>
        <exclude>com/codigo/semana7/application/service/**</exclude>
        <exclude>com/codigo/semana7/domain/model/**</exclude>
        <exclude>com/codigo/semana7/domain/ports/**</exclude>
        <exclude>com/codigo/semana7/infrastructure/config/**</exclude>
        <exclude>com/codigo/semana7/infrastructure/controller/**</exclude>
        <exclude>com/codigo/semana7/infrastructure/entity/**</exclude>
        <exclude>com/codigo/semana7/EjemploDeHexagonalApplication.*</exclude>
    </excludes>
</configuration>
```

Estructura para crear el TEST

Lo recomendable para crear el Test, es tener la misma estructura de Paquete de nuestras SRC





Reporte Jacoco sin Test

Ejemplo de Hexagonal > com.codigo.semana7.application.usecase > UsuarioUseCaseImpl

UsuarioUseCaseImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
updateUsuario(Long, Usuario)		0 %		n/a	1	1	1	1	1	1
crearUsuario(Usuario)		0 %		n/a	1	1	1	1	1	1
getUsuario(Long)		0 %		n/a	1	1	1	1	1	1
deleteUsuario(Long)		0 %		n/a	1	1	1	1	1	1

com.codigo.semana7.infrastructure.repositori

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
[illegible]				0 %	6	7	13	16	4	5	0	1
PersonaJpaRepositoryAdapter		100 %		100 %	0	7	0	16	0	5	0	1
Total	53 of 118	55 %	4 of 8	50 %	6	14	13	32	4	10	0	2

Creación del Ejecicio

Practica en Clase

Concluciones